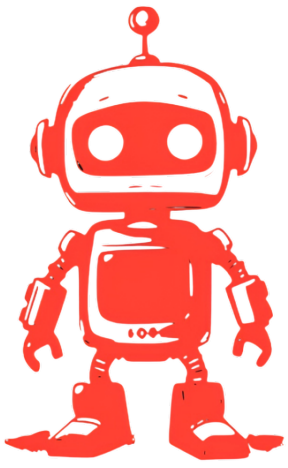
Architectural Requirements



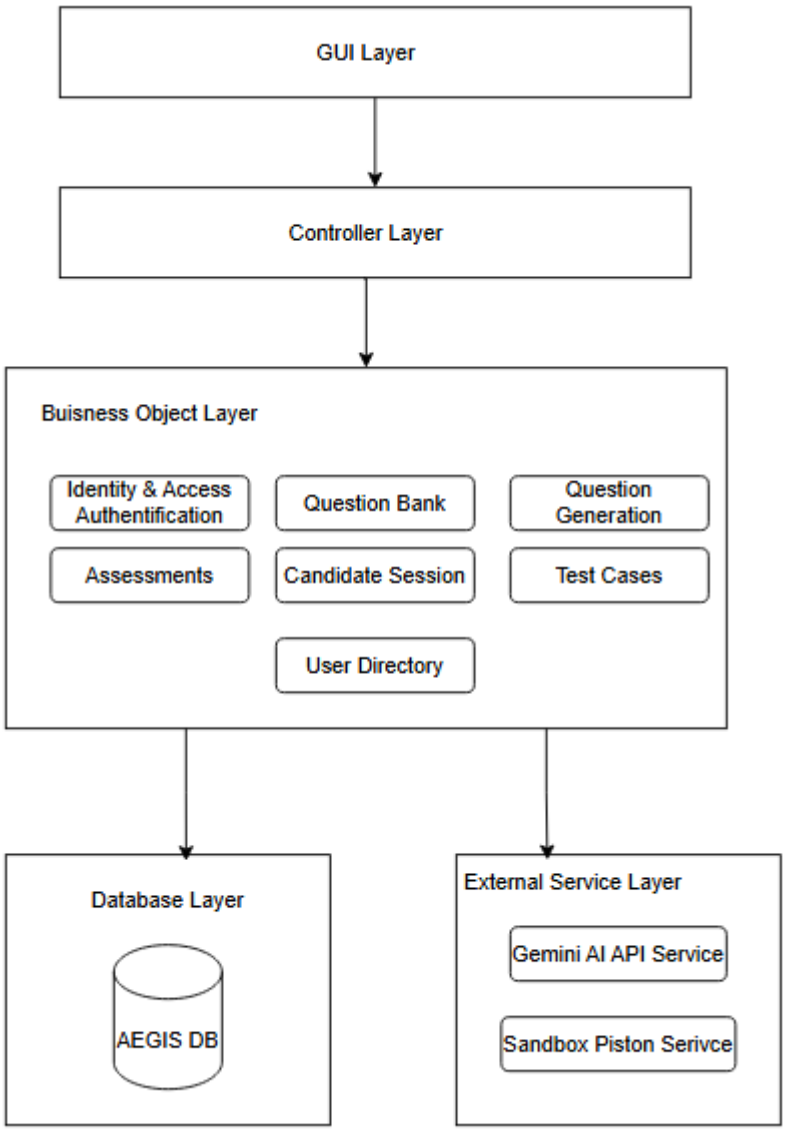
AEGIS

bbd

bbdsoftware.com

Contents

Architectural Diagram



Architectural Patterns and Justifications

Architectural Pattern	Justification
N-Tier Architecture	The Controller, Business Objects, and Database layers are kept strictly separated, no route handler issues a database call directly. Every request is delegated down through exactly one Business Objects component before reaching the Database Layer.
Client-server Architecture	The browser client and backend server exchange requests and responses in a client server relationship, with the server holding all business logic and data access. This same relationship recurs one level deeper: the backend itself acts as a client to two further servers: the external AI model and the sandboxed code execution service, waiting on and executing their responses the same way the browser waits on the backend's.
No Object-Persistence Framework	The system has exactly one data store, so the added indirection of a persistence framework designed to hide *several* databases would not currently earn its cost.
Monorepo Build with Deployment Seperation	The backend and frontend maintain separate: dependency manifests, CI pipelines that only act on their own directory. For our backend we also use a dedicated deployment container. This design decision allows for the halves to be able to build and deploy independently, thus allowing the team to work in parallel.

Architectural Constraints

Client Requirement Constraints

1. Exactly two user roles - Candidate and Recruiter, govern all access control in the system.
2. The platform must comply with South Africa's Protection of Personal Information Act (POPIA), given that it processes candidate identifiable data; candidate identifiable information may not be shared with third-party AI providers without appropriate safeguards.
3. Zero Setup - No additional installation or configuration required; the service must be usable immediately upon access.

Deployment Constraints

1. Budget Restriction: must make use of free-tier, open source or relatively inexpensive solutions.
2. Fixed AI Model: we make use of gemini 3.1 flash-lite for our prompting and as such the service is susceptible to the deprecation of the API service should Google decide such.
3. High Availability: the app must maintain at least 90% uptime.
4. Piston container dependency: The sandboxed code execution service requires a container runtime and privileged container access and is disabled by default in local development.

Technical Constraints

1. External Service Integration: both external service integrations (the AI model and the sandbox) communicate over synchronous HTTP request/response only; there is no asynchronous, queued, or event driven processing path for either, meaning a slow or unresponsive dependency directly ties up the request that called it.
2. Data Access: access is constrained to a single relational database schema, reached only through the project's ORM session - there is no abstraction layer supporting multiple or alternate data stores.

Design Patterns

1. **Singleton Pattern**

   Purpose:

   Ensure the application's configuration and database connection are constructed once and shared everywhere, as opposed to being rebuilt on every use.

   Application:

   - Settings: Instantiated once at module load, every route, service, and utility module imports the same object by reference rather than constructing its own.
   - DB Engine and Session Factory: Built once on top of the shared settings object, a new session is created per request, but always against the single shared engine and connection pool.

   Benefits:

   - One consistent, fully initialised configuration object across every layer.
   - Avoids the cost (and potential bugs risk) of rebuilding the database engine on every request.

2. **State Pattern** (Candidate Assessment Lifecycle)

   Purpose:

   Model the lifecycle of a candidate's assessment session, restricting which transitions are allowed from which status.

   Application:

   - **States:** Started, In Progress, Completed, Expired. (Stored as enumerated columns)
   - **Started - In Progress:** Guarded, explicitly rejected if the session is already In Progress, Completed, or Expired.
   - **Completed:** Unguarded, set unconditionally, with no check on the session's current status first.

   Benefits:

   - Prevents re-entering (and resetting the timer on) an already started or completed session, on the one guarded path.
   - Eliminates large conditional blocks.

MAPPING QUALIFY REQUIREMENTS TO ARCHITECTURAL DECISIONS

Quality Requirements	Architectural Decision
Performance: The system must handle concurrent candidate load without crashing and ensure minimal lag during simultaneous access	FastAPI's asynchronous request handling allows multiple requests to be processed concurrently without blocking, ensuring the system remains responsive when multiple candidates are completing assessments simultaneously
Usability: The system must handle errors gracefully with clear messaging and provide an accessible user interface	FastAPI returns structured HTTP error responses with descriptive detail messages for every failure case. Pydantic validates all incoming request payloads at the API boundary before any business logic runs, catching malformed data early and returning field-level error messages.
Reliability: The system must recover from crashes without data loss.	Supabase's managed PostgreSQL service includes automated daily backups, reducing data loss risk in the event of a critical failure.
Security: The system must strictly enforce access levels so candidates only see their assigned assessments and only recruiters have admin privileges	JWT role-based authentication is enforced on every protected API endpoint.
Maintainability: The system must be modular with low coupling between components and code must adhere to industry standard quality principles.	The three-tier layered architecture strictly separates the presentation layer (Next.js), application layer (FastAPI), and data layer (Supabase PostgreSQL), ensuring changes in one layer do not cascade into others.